Rockchip RK3588 Linux Robot SDK Quick Start

ID: RK-JC-YF-H52

Release Version: V0.0.2

Release Date: 2025-12-05

Security Level: ☐Top-Secret ☐Secret ☐Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2025. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document primarily describes the basic usage of the RK3588 Linux Robot SDK, aiming to help developers quickly understand and use the SDK development package.

Target Audience

This document (this guide) is mainly suitable for the following engineers:

Technical Support Engineers

Software Development Engineers

Chip System Support Status

Chip Name	Kernel Version	Debian Version	Buildroot Version
RK3588 Series	6.1	11	2024.02

Revision History

Date	Version	Author	Modification Description
2025-10-20	V0.0.1	WJL	Alpha version.
2025-12-05	V0.0.2	WJL	Add Robotic EVB instructions.

Table of Contents

Rockchip RK3588 Linux Robot SDK Quick Start
1. Precompiled Firmware
2. Set up a Development Environment
   2.1 Preparing the development environment
   2.2 Install libraries and toolsets
      2.2.1 Check and upgrade the `python` version of the host
      2.2.2 Check and Upgrade the `make` Version on the Host
      2.2.3 Check and Upgrade the `lz4` Version on the Host
   2.3 Set up a Docker environment
3. Software Developer Guide
   3.1 Developer Guide
   3.2 Chip Information
   3.3 Buildroot Developer Guide
   3.4 Debian Developer Guide
   3.5 RKNPU Developer Guide
   3.6 RK182X Developer Guide
   3.7 RT-Linux Developer Guide
   3.8 AMP Developer Guide
   3.9 EtherCAT Developer Guide
   3.10 Software Update History
4. Hardware Developer Guide
5. SDK Configuration Framework Description
   5.1 SDK Project Directory Introduction
   5.2 RKROS Project Directory Introduction
6. SDK Compilation Instructions
   6.1 Viewing SDK Compilation Commands
   6.2 SDK Board-Level Configurations
   6.3 SDK Customized configurations
   6.4 Fully Automated Compilation
   6.5 Module Compilation
      6.5.1 U-boot Compilation
      6.5.2 Kernel Compilation
      6.5.3 Recovery Compilation
      6.5.4 Debian Compilation
      6.5.5 RKROS Compilation
      6.5.6 Firmware Packaging
7. Flashing Instructions
   7.1 Windows Flashing Instructions
   7.2 Linux Flashing Instructions
   7.3 System Partition Description

1. Precompiled Firmware

To help developers quickly complete relevant evaluation, comparison, and development, we provide precompiled firmware for the RK3588 Robot development board.

The firmware is available for download at the following official website precompiled firmware.
Linux6.1 firmware path: **Linux Robot SDK Firmware** -> **Linux6.1** -> **RK3588**

For SDK customization or a quick start, please refer to the following chapters.

2. Set up a Development Environment

2.1 Preparing the development environment

It is recommended to use Ubuntu22.04 and above systems for compilation. Other Linux distributions require corresponding adjustments to the software package.

Software requirements: Ubuntu22.04 and above.
Hardware requirements: Disk space greater than 100GB. Doing multiple builds requires more disk space.

2.2 Install libraries and toolsets

Use the `apt` command to install the libraries and toolsets required to compile the SDK:

```
sudo apt install -y \
bash-completion bc bison bsdmainutils build-essential ccache chrpath cmake \
cpio curl device-tree-compiler diffstat dos2unix expat expect fdisk file \
flex gawk g++-multilib gcc-arm-none-eabi gcc-multilib genext2fs gettext \
git iputils-ping libegl1-mesa libelf-dev libgmp-dev libgucharmap-2-90-dev \
liblz4-tool libmpc-dev libncurses-dev libsdl1.2-dev libssl-dev libtinfo5 \
live-build locales mesa-common-dev mtd-utils mtools net-tools patchelf \
python2 python2-dev python3 python3-git python3-jinja2 python3-pexpect \
python3-pip python3-pyelftools python3-subunit python-is-python3 \
qemu-user-static repo rsync ruby scons socat ssh strace sudo swig \
texinfo time tree u-boot-tools unzip vim
```

> Description:
> The installation command is applicable to Ubuntu22.04. For other versions, please use the corresponding installation command according to the name of the installation package. If you encounter an error when compiling, you can install the corresponding software package according to the error message.
>
> - Python 3.6 or later versions is required to be installed, and python 3.6 is used as an example here.
> - make requires make 4.0 and above to be installed, take make 4.2 as an example here.
> - lz4 1.7.3 or later versions is required to be installed.

2.2.1 Check and upgrade the `python` version of the host

The method of checking and upgrading the `python` version of the host is as follows:

- Check host `python` version

```
$ python3 --version
Python 3.10.6
```

If you do not meet the requirements of python>=3.6 version, you can upgrade it in the following way:

- Upgrade `python 3.6.15` new version

```
PYTHON3_VER=3.6.15
echo "wget
https://www.python.org/ftp/python/${PYTHON3_VER}/Python-${PYTHON3_VER}.tgz"
echo "tar xf Python-${PYTHON3_VER}.tgz"
echo "cd Python-${PYTHON3_VER}"
echo "sudo apt-get install libsqlite3-dev"
echo "./configure --enable-optimizations"
echo "sudo make install -j8"
```

2.2.2 Check and Upgrade the `make` Version on the Host

The method to check and upgrade the `make` version on the host is as follows:

- Check the `make` version on the host

```
$ make -v
GNU Make 4.2
Built for x86_64-pc-linux-gnu
```

- Upgrade to the new version of `make 4.2`

```
$ sudo apt update && sudo apt install -y autoconf autopoint

git clone https://gitee.com/mirrors/make.git
cd make
git checkout 4.2
git am $BUILDROOT_DIR/package/make/*.patch
autoreconf -f -i
./configure
make make -j8
sudo install -m 0755 make /usr/bin/make
```

2.2.3 Check and Upgrade the `lz4` Version on the Host

The method to check and upgrade the `lz4` version on the host is as follows:

- Check the `lz4` version on the host

```
$ lz4 -v
*** LZ4 command line interface 64-bits v1.9.3, by Yann Collet ***
refusing to read from a console
```

- Upgrade to the new version of `lz4`

```
git clone https://gitee.com/mirrors/LZ4_old1.git
cd LZ4_old1

make
sudo make install
sudo install -m 0755 lz4 /usr/bin/lz4
```

2.3 Set up a Docker environment

To help developers quickly prepare the compilation environment, we provide docker images and image production methods, which can be obtained from the official website Docker image.

- docker-sdk: built on amd64/ubuntu22.02 for compiling generic Linux firmware.
- docker-ros: built on arm64v8/debian11 for cross-architecture compilation of ROS packages.

Docker instructions can be found in the following documents:

```
<SDK>/docs/en/Linux/Docker/
└── Rockchip_Developer_Guide_Linux_Docker_Deploy_EN.pdf
```

3. Software Developer Guide

3.1 Developer Guide

To help developers quickly get started with SDK development and debugging, please refer to the documentation:

```
<SDK>/docs/en/RK3588/
└── Rockchip_Developer_Guide_Linux_Software_EN.pdf
```

3.2 Chip Information

To help developers quickly get started with the development and debugging of RK3588 and RK3588S, please refer to the documentation:

```
<SDK>/docs/en/RK3588/Datasheet/
├── Rockchip_RK3588_Datasheet_V2.0-20250425.pdf
...
└── Rockchip_RK3588S2_Datasheet_V1.2-20241212.pdf
```

3.3 Buildroot Developer Guide

To help developers quickly get started with the development and debugging of the Buildroot system, please refer to the documentation:

```
<SDK>/docs/en/Linux/System/
└── Rockchip_Developer_Guide_Buildroot_EN.pdf
```

3.4 Debian Developer Guide

To help developers quickly get started with Debian system development and debugging, please refer to the documentation:

```
<SDK>/docs/en/Linux/System/
└── Rockchip_Developer_Guide_Debian_EN.pdf
```

3.5 RKNPU Developer Guide

SDK provides RKNPU development tools:

- RKNN-TOOLKIT2: A development kit for RKNN model generation and evaluation on PC, available from the `<SDK>/external/rknn-toolkit2` directory.
- RKNN API: Used for inference RKNN model generated by RKNN-TOOLKIT2, available from the `<SDK>/external/rknpu2` directory.

For specific instructions, please refer to the documentation:

```
<SDK>/external/rknn-toolkit2/doc/
├── 01_Rockchip_RKNPU_Quick_Start_RKNN_SDK_V2.3.2_EN.pdf
...
├── 04_Rockchip_RKNPU_API_Reference_RKNNRT_V2.3.2_EN.pdf
└── 05_RKNN_Compiler_Support_Operator_List_V2.3.2.pdf
```

3.6 RK182X Developer Guide

The RK182X series coprocessors feature high computing power, high bandwidth, and low latency, and focus on high-performance neural network inference tasks. Supports model conversion, inference and performance evaluation of deep learning frameworks such as PyTorch and ONNX.

Developers can contact the business to obtain relevant SDKs.

3.7 RT-Linux Developer Guide

Products with high real-time performance requirements can use the PREEMPT_RT or Xenomai solutions to improve real-time performance.
For specific instructions, please refer to the documentation:

```
<SDK>/docs/Patches/Real-Time-Performance/
├── Rockchip_Develop_Guide_Linux_RealTime_EN.md
├── Rockchip_Develop_Guide_Linux_RealTime_Performance_Test_Report_EN.pdf
└── Rockchip_Trouble_Shooting_Linux_Real-Time_Performance_EN.pdf
```

3.8 AMP Developer Guide

For products with higher requirements on real-time performance, you can consider using the AMP solution. Please refer to the documentation:

```
<SDK>/docs/en/Common/AMP/
└── Rockchip_Developer_Guide_AMP_EN.pdf
```

3.9 EtherCAT Developer Guide

To help developers quickly get started with the development and debugging of the EtherCAT master protocol, please refer to the documentation:

```
<SDK>/docs/en/Linux/ApplicationNote/
└── Rockchip_Use_Guide_Linux_EtherCAT_IgH_EN.pdf
```

3.10 Software Update History

The software release version can be viewed through the project xml. The specific method is as follows:

```
.repo/manifests$ realpath rk3588_linux6.1_robot_release.xml
# For example: printf version v0.0.2, update time on 20251205
# <SDK>/.repo/manifests/release/rk3588_linux6.1_robot_alpha_v0.0.2_20251205.xml
```

Software release version upgrade and update content can be viewed through the project text, refer to the project directory:

```
<SDK>/.repo/manifests/RK3588_Linux6.1_Robot_SDK_Release_Note.md
```

4. Hardware Developer Guide

RK3588 hardware design and user guide, please refer to the documentation:

```
<SDK>/docs/en/RK3588/Hardware/
...
├── Rockchip_RK3588_EVB1_User_Guide_V1.2_EN.pdf
├── Rockchip_RK3588_EVB7_User_Guide_V1.0_EN.pdf
└── Rockchip_RK3588_Hardware_Design_Guide_V1.4_EN.pdf
```

5. SDK Configuration Framework Description

5.1 SDK Project Directory Introduction

There are buildroot, debian, device, docs, kernel, robot, u-boot and other directories in the project directory. A manifest is used to manage repositories, and the repo tool is used to manage each directory or its subdirectories corresponding to git projects.

- buildroot: Root filesystem developed based on Buildroot 2024.02.
- debian: Root filesystem developed based on Debian bullseye.
- device/rockchip: Contains chip-level board configurations as well as scripts and files for compiling and packaging firmware.
- docs: Contains development guidance documents, platform support lists, tool usage documents, Linux development guides, etc.
- external: Contains third-party related repositories, including display, audio/video, camera, network, security, etc.
- hal: Contains bare-metal development libraries based on RK HAL hardware abstraction layer, used for AMPAK solutions.
- kernel: Contains the code for Kernel development.
- output: Contains information about each generated firmware, compilation information, XML, host environment, etc.
- prebuilts: Contains the cross-compilation toolchain.
- rkbin: Contains Rockchip related binaries and tools.
- robot: Contains robot-specific acceleration algorithm libraries, model libraries, rkros codes, etc.
- rockdev: Contains compiled output firmware, actually a symlink to output/firmware.
- tools: Contains commonly used tools under Linux and Windows operating systems.
- u-boot: Contains U-Boot code.
- uefi: Contains UEFI code based on edk2 V2.7 version development.

5.2 RKROS Project Directory Introduction

The RKROS project is located in the `<SDK>/robot/` directory and includes common_algorithm, docs, model_zoo, prebuilts, rkros, etc. The specific project description is as follows:

- common_algorithm: Contains the hardware acceleration algorithm library and supports function interfaces such as PCL and IDC.
- docs: Contains RKROS related development documents.
- model_zoo: Contains the model library commonly used by robots.
- prebuilts: Contains precompiled packages.
- rkros: Contains rkros code and is the default workspace of rkros.
- scripts: Contains scripts that compile and package rkros.

6. SDK Compilation Instructions

SDK can configure and compile related functions through `./build.sh` with target parameters. Please refer to the compilation instructions of `<SDK>/device/rockchip/common/README.md` for details.

6.1 Viewing SDK Compilation Commands

Compile related functions by running `./build.sh <target>`, and you can view the specific compilation commands through `./build.sh help`.

```
############### Rockchip Linux SDK ###############

Manifest: rk3588_linux6.1_robot_alpha_v0.0.2_20251205.xml

Log colors: message notice warning error fatal

Usage: build.sh [OPTIONS]
Available options:
chip[:<chip>[:<config>]]            choose chip
defconfig[:<config>]                choose defconfig
config                              modify SDK defconfig
...
updateimg                           build update image
otapackage                          build A/B OTA update image
all                                 build images
release                             release images and build info
save                                alias of release
all-release                         build and release images
allsave                             alias of all-release
shell                               setup a shell for developing
cleanall                            cleanup
clean[:module[:module]]...          cleanup modules
post-rootfs <rootfs dir>            trigger post-rootfs hook scripts
```

```
help                                usage

Default option is 'all'.

Available options for RKROS:

rkros[:debug|:release]                  build all rkros packages
rkros[:cleanall]                        cleanup all rkros packages
rkros[:<pkg>-[debug|release]]           build the specified rkros package
  supported rkros packages:
...
```

6.2 SDK Board-Level Configurations

Navigate to the project directory `<SDK>/device/rockchip/rk3588`:

Board-Level Configuration	Description
rockchip_rk3588_evb1_lp4_v10_defconfig	Suitable for RK3588 EVB1 development board with LPDDR4
rockchip_rk3588_evb7_v11_defconfig	Suitable for RK3588 EVB7 development board with LPDDR4
rockchip_rk3588_evb10_v10_defconfig	Suitable for RK3588 EVB10 development board with LPDDR4
rockchip_rk3588_robotic_evb_v11_defconfig	Suitable for RK3588 Robotic EVB development board with LPDDR4
rockchip_rk3588s_evb1_lp4x_v10_defconfig	Suitable for RK3588S EVB1 development board with LPDDR4
rockchip_defconfig	Default configuration, which will be symbolically linked to a specific board-level configuration

Method 1:
Add the board-level configuration file after `./build.sh`, for example:

Select the board-level configuration **Suitable for RK3588 Robotic EVB development board with LPDDR4**:

```
./build.sh device/rockchip/rk3588/rockchip_rk3588_robotic_evb_v11_defconfig
```

Method 2:

Select configuration by running `./build.sh lunch`

```
./build.sh lunch
Pick a defconfig:

1. rockchip_defconfig
7. rockchip_rk3588_robotic_evb_v11_defconfig
...
Which would you like? [1]:
```

6.3 SDK Customized configurations

The SDK can be configured through `./build.sh menuconfig`, and the main components that can be configured are as follows:

```
(rockchip_rk3588_robotic_evb_v11_defconfig) Name of defconfig to save
[*] Rootfs (Buildroot|Debian|Yocto)  --->
[*] Loader (U-Boot)  --->
[ ] AMP (Asymmetric Multi-Processing System)
[*] Kernel (Embedded in an Android-style boot image)  --->
    Boot (Android-style boot image)  --->
[*] Recovery (based on Buildroot)  --->
    *** Security feature depends on buildroot rootfs ***
    Extra partitions (oem, userdata, etc.)  --->
    Firmware (partition table, misc image, etc.)  --->
[*] Update (Rockchip update image)  --->
    Others configurations  --->
```

- Rootfs: Root file system, here you can choose different root file system configurations such as Buildroot and Debian.
- Loader (U-Boot): This is the configuration for the bootloader, typically U-Boot, which is used to initialize hardware and load the main operating system.
- AMP: A multi-core heterogeneous boot solution suitable for application scenarios requiring real-time performance.
- Kernel: Here you configure kernel options to customize the Linux kernel according to your hardware and application needs.
- Boot: Here you configure the support format for the Boot partition.
- Recovery (based on buildroot): This is the configuration for the recovery environment based on buildroot, used for system recovery and upgrades.
- PCBA test (based on buildroot): This is the configuration for a PCBA (Printed Circuit Board Assembly) test environment based on buildroot.
- Security: Enabling security features, including Secureboot methods, Optee storage methods, and writing Keys.
- Extra partitions: Used to configure additional partitions.
- Firmware: Here you configure firmware-related options.
- Update (Rockchip update image): Used to configure options for Rockchip's complete firmware.
- Others configurations: Other additional configuration options.

The `make menuconfig` interface provides a text-based user interface to select and configure various options. Once configuration is complete, use the command `make savedefconfig` to save these settings, so that custom builds will be done based on these settings.

With the above configs, you can choose different Rootfs/Loader/Kernel configurations for various custom builds, allowing flexible selection and configuration of system components to meet specific needs.

6.4 Fully Automated Compilation

To ensure smooth updates of the Software Development Kit (SDK) each time, it is recommended to clean previous compilation products before updating. This helps avoid potential compatibility issues or compilation errors, as old compilation products may not be suitable for the new version of the SDK. To clean these compilation products, run the command `./build.sh cleanall`.

Enter the root directory of the project and execute the following command to automatically complete all compilations:

```
./build.sh all # Compiles module code only (u-Boot, kernel, Rootfs, Recovery)
               # Then run `./build.sh ./mkfirmware.sh` to package the firmware

./build.sh     # Compiles module code (u-Boot, kernel, Rootfs, Recovery)
               # Packages into update.img complete upgrade package
               # Copies all compilation information and generates it in the out
directory
```

6.5 Module Compilation

6.5.1 U-boot Compilation

```
./build.sh uboot
```

6.5.2 Kernel Compilation

Method 1:

```
./build.sh kernel
```

Method :

```
cd kernel
export CROSS_COMPILE=../prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-
x86_64-aarch64-none-linux-gnu/bin/aarch64-none-linux-gnu-
make ARCH=arm64 rockchip_linux_defconfig rk3588.config rk3588_linux.config
rk3588_robotic.config
make ARCH=arm64 rk3588-robotic-evb-v11-linux.img -j
```

Method 3:

```
cd kernel
export CROSS_COMPILE=aarch64-linux-gnu-
make ARCH=arm64 rockchip_linux_defconfig rk3588.config rk3588_linux.config
rk3588_robotic.config
make ARCH=arm64 rk3588-robotic-evb-v11-linux.img -j
```

6.5.3 Recovery Compilation

Navigate to the root directory of the project and execute the following command to automatically complete the compilation and packaging of recovery.

```
./build.sh recovery
```

After compilation, a recovery.img will be generated in the Buildroot directory `output/rockchip_rk3588_recovery/images`.

> Note: The recovery.img contains the kernel, so every time the kernel changes, recovery.img needs to be repackaged. The method to repackage recovery.img is as follows:

```
source buildroot/envsetup.sh
cd buildroot
make recovery-reconfigure
cd -
./build.sh recovery
```

> Note: Recovery is a non-essential feature, and some board-level configurations may not set it up.

6.5.4 Debian Compilation

```
./build.sh debian
```

After compilation, a `linaro-rootfs.img` file will be generated in the directory `<SDK>/debian`.

> Note: It is necessary to install the relevant dependency packages beforehand:
> sudo apt-get install binfmt-support qemu-user-static live-build
> sudo dpkg -i ubuntu-build-service/packages/*
> sudo apt-get install -f

6.5.5 RKROS Compilation

Compile all code under the rkros workspace:

```
./build.sh rkros:debug
or
./build.sh rkros:release
```

Compile the code specified in the rkros workspace:

```
./build.sh rkros:rkros_examples-debug
or
./build.sh rkros:rkros_examples-release
```

> **debug**: If install_base is not specified, the compiled product is in the `<SDK>/robot/rkros/install` directory.
>
> **release**: Specify install_base to `/opt/rkros/packagename`, automatically packaged into deb, compiled products in the `<SDK>/robot/rkros/output` directory.

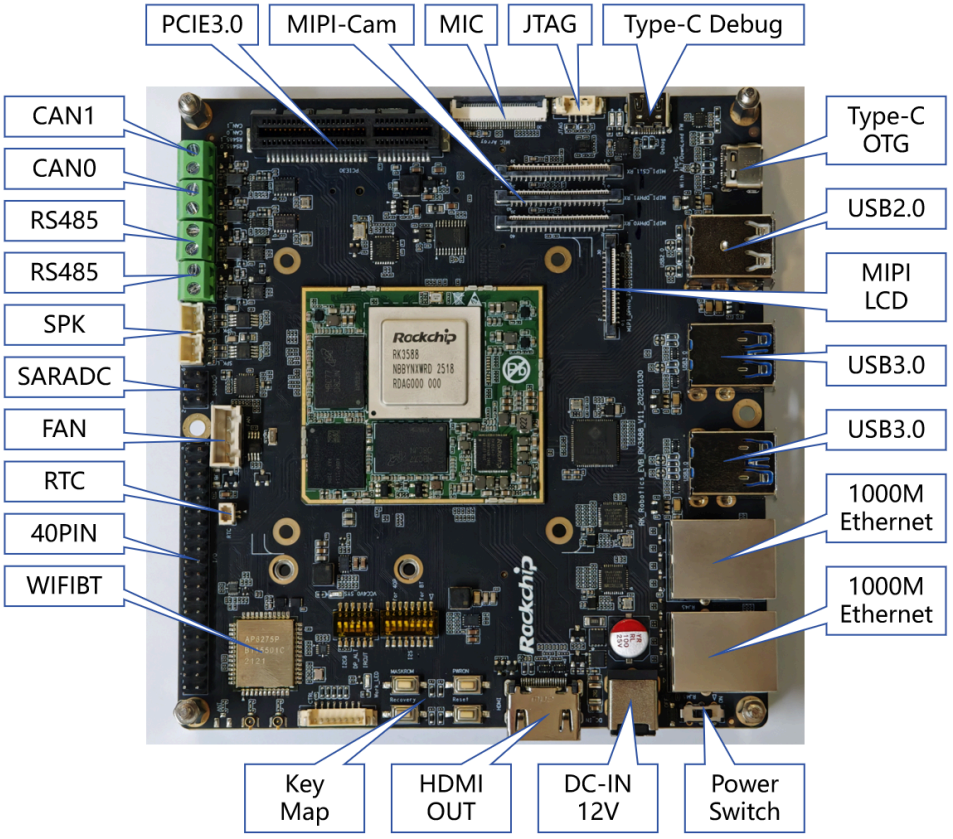
6.5.6 Firmware Packaging

After completing the compilation of kernel, u-boot, recovery, and rootfs, execute the following command to package the firmware into the `<SDK>/output/firmware` directory:

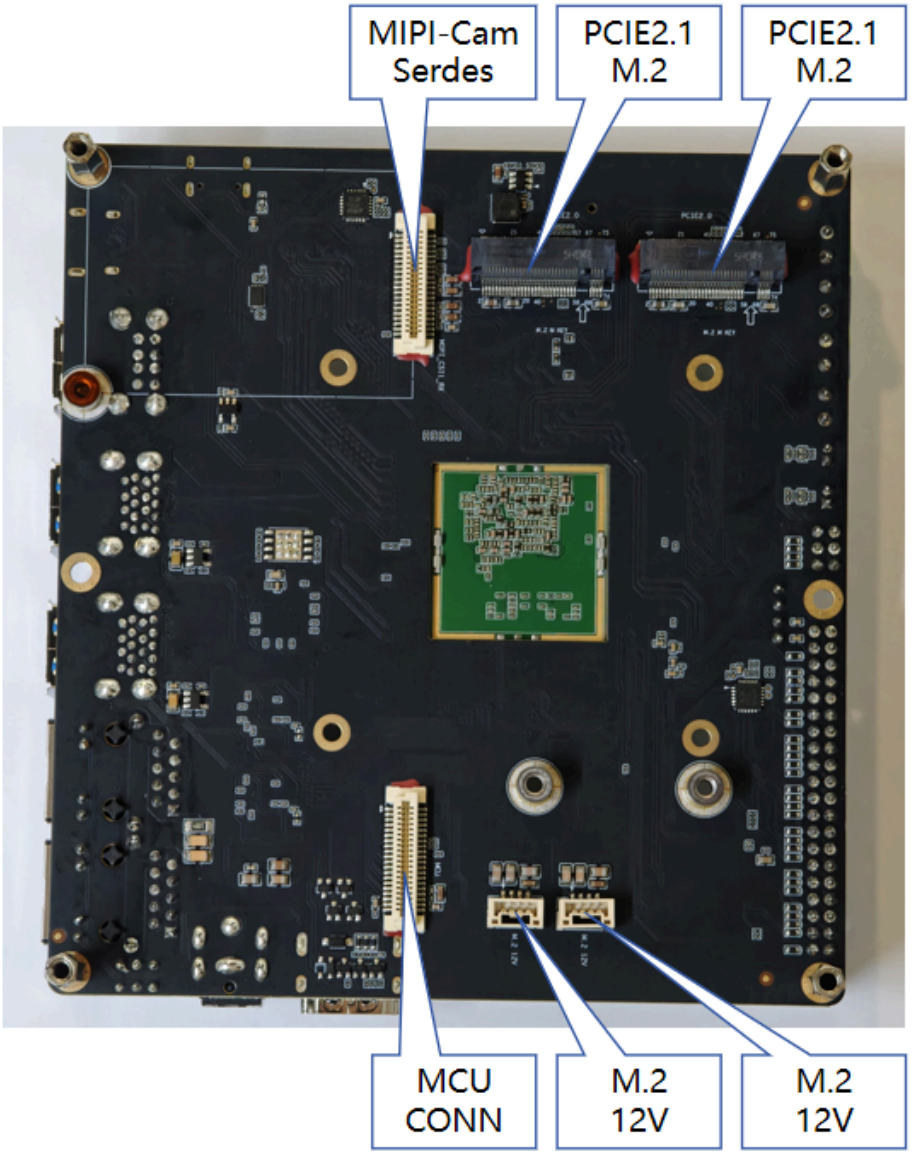
```
./build.sh firmware
```

7. Flashing Instructions

The interface layout on the front side of the RK3588 Robotic EVB development board is as follows:



The interface layout on the back side of the RK3588 Robotic EVB development board is as follows:

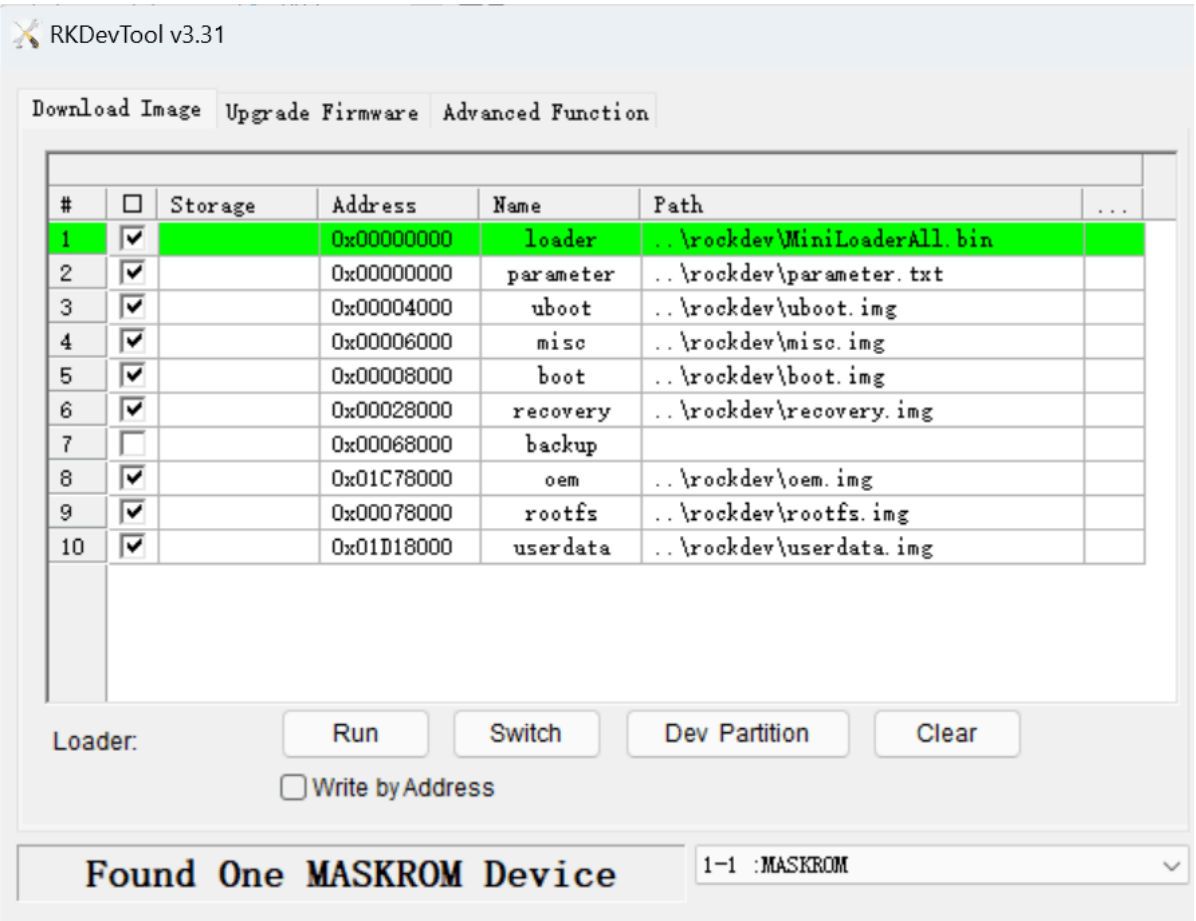


7.1 Windows Flashing Instructions

The SDK provides a Windows flashing tool (the tool version must be V3.31 or higher), which is located in the root directory of the project:

```
<SDK>/tools/windows/RKDevTool
```

As shown in the figure below, after compiling the corresponding firmware, the device needs to enter the MASKROM or BootROM flashing mode. Connect the USB download cable and hold the `MASKROM` button without releasing, then press and release the reset button `RST` to enter the MASKROM mode. Load the corresponding path of the compiled firmware and click `Execute` to perform the flashing. Alternatively, hold the `recovery` button without releasing and press the reset button `RST` to enter the loader mode for flashing. Below are the partition offsets for MASKROM mode and the flashing files.
(Note: The Windows PC needs to run the tool with administrator privileges to execute)



Note: Before flashing, the latest USB driver must be installed. For more details on the driver, see:

```
<SDK>/tools/windows/
└──DriverAssitant_v5.13.zip
```

7.2 Linux Flashing Instructions

The flashing tool for Linux is located in the `<SDK>/tools/linux/Linux_Upgrade_Tool` directory (the tool version must be V2.36 or higher). Please ensure that your board is connected to MASKROM/LOADER rockusb. For example, if the firmware compiled is in the `<SDK>/rockdev` directory, the upgrade commands are as follows:

```
sudo ./upgrade_tool ul rockdev/MiniLoaderAll.bin -noreset
sudo ./upgrade_tool di -p rockdev/parameter.txt
sudo ./upgrade_tool di -u rockdev/uboot.img
sudo ./upgrade_tool di -misc rockdev/misc.img
sudo ./upgrade_tool di -b rockdev/boot.img
sudo ./upgrade_tool di -recovery rockdev/recovery.img
sudo ./upgrade_tool di -oem rockdev/oem.img
sudo ./upgrade_tool di -rootfs rocdev/rootfs.img
sudo ./upgrade_tool di -userdata rockdev/userdata.img
sudo ./upgrade_tool rd
```

Or upgrade the packaged complete firmware:

```
sudo ./upgrade_tool uf rockdev/update.img
```

Or in the root directory, when the machine is running in MASKROM state, run the following upgrade:

```
./rkflash.sh
```

7.3 System Partition Description

Default Partition Description (Below is the RK3588 Robotic EVB Partition Reference)

Number	Start (sector)	End (sector)	Size	Name
1	16384	24575	4 MB	uboot
2	24576	32767	4 MB	misc
3	32768	163839	64 MB	boot
4	163840	425983	128 MB	recovery
5	425984	491519	32 MB	bakcup
6	491520	105349120	50 GB	rootfs
7	105349121	105611265	128 MB	oem
8	105611265	122142686	7.8 GB	userdata

- The uboot partition: for the uboot.img compiled from uboot.
- The misc partition: for misc.img, used by recovery.
- The boot partition: for boot.img compiled from the kernel.
- The recovery partition: for recovery.img compiled from recovery.
- The backup partition: reserved, not in use at the moment.
- The rootfs partition: for rootfs.img compiled from buildroot, debian, or yocto.
- The oem partition: for the manufacturer's use, storing manufacturer's apps or data. Mounted in the /oem directory.
- The userdata partition: for temporary file generation by apps or for final user use. Mounted in the /userdata directory.